Estructuras definidas por el usuario en JavaScript

Caso práctico



Ministerio de Educación (Uso educativo nc)

Antonio ha avanzado mucho en su estudio de JavaScript, viendo los principales objetos con los que puede trabajar, sus métodos y propiedades.

Ha llegado el momento, de ver cómo puede organizar los datos que genera en su aplicación de JavaScript, de forma que le permita gestionarlos más eficientemente.

También verá cómo crear funciones, el alcance de las variables y cómo utilizarlas en su código. Y para terminar **Juan**, le va a explicar un poco más en detalle, cómo podrá crear nuevos objetos y asignarles propiedades y métodos en JavaScript.

Antonio ha comenzado a analizar más a fondo parte del trabajo que tiene que realizar, y comenta con su directora **Ada**, la posibilidad de hacer algunos bocetos de las innovaciones y mejoras que quiere aportar al proyecto. **Ada** está muy contenta con los progresos de **Antonio** y está deseando ver los bocetos.

En esta unidad de trabajo, primero profundizaremos en las funciones, revisando las múltiples formas de declararlas, el ámbito y encapsulamiento, junto con los nuevos añadidos introducidos a lo largo de la evolución de ECMAScript.

En segundo lugar, se introducirá el concepto de estructura de datos, centrándonos en la estructura de datos Array (cómo utilizarlo, funciones de array, copias, etc.), Map (también conocidos como diccionarios o tablas hash) y Sets.

En tercer lugar, revisaremos el concepto de clase y objeto definidos por el usuario, con sus propias propiedades y métodos. Esto es fundamental, ya que las clases son un elemento clave en patrones de diseño por componentes, como los que utiliza React.

Finalmente, revisaremos los principales patrones de diseño que se utilizan en JavaScript, tanto en la lógica del negocio como en la interfaz de usuario, y cómo estos patrones pueden mejorar la estructura y mantenibilidad del código.

Recomendación

En esta unidad no se hace un uso intensivo del DOM, ya que nos centraremos en crear estructuras de datos, clases y funciones. Por lo tanto, puedes crear un archivo main.js y probarlo utilizando Node: `node ./main.js`, o probarlo directamente desde la consola de Node o del navegador.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Ministerio de Educación, Formación Profesional y Deportes (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.- Funciones

Caso práctico



Ministerio de Educación (Uso educativo nc)

Juan está siguiendo los progresos de **Antonio**, y le recomienda empezar a ver funciones. Las funciones son una herramienta muy potente en los lenguajes de programación, ya que le van a permitir realizar tareas de una manera mucho más organizada, y además le permitirán reutilizar un montón de código en sus aplicaciones.

Antonio verá cómo crear funciones, cómo pasar parámetros, el ámbito de las variables dentro de las funciones, cómo anidar funciones y las funciones predefinidas en JavaScript. **Antonio** está muy ilusionado con este tema, ya que a lo largo de las unidades anteriores ha estado utilizando funciones y es ahora el momento en el que realmente verá toda la potencia que le pueden aportar a sus aplicaciones.

En unidades anteriores ya has visto y utilizado alguna vez funciones. **Una función es la definición de un conjunto de acciones pre-programadas**. Las funciones se llaman a través de eventos o bien mediante comandos desde nuestro script.

En el siguiente apartado detallaremos todo lo que debes tener en cuenta para trabajar con funciones. ¡Vamos a ello!

Autoevaluación

En JavaScript no podemos definir nuevas funciones y debemos atenernos a las que ya vienen definidas por defecto en el lenguaje. ¿Verdadero o falso?

○ Verdadero ○ Falso

Falso

En JavaScript sí podemos definir nuevas funciones y, a su definición y uso es a lo que nos dedicaremos en este apartado.

1.1.- Uso básico de funciones



Everaldo Coelho (GNU/GPL)

Siempre que sea posible, tienes que diseñar funciones que puedas reutilizar en otras aplicaciones, de esta forma, tus funciones se convertirán en pequeños bloques constructivos que te permitirán ir más rápido en el desarrollo de nuevos programas.

Si conoces otros lenguajes de programación, quizás te suene el término de *método o procedimiento*. En JavaScript no vamos a distinguir entre **procedimientos** (que ejecutan acciones), o **funciones** (que ejecutan acciones y devuelven valores). **En JavaScript siempre se llamarán funciones**.

Una función es capaz de devolver un valor a la instrucción que la invocó, pero esto no es un requisito obligatorio en JavaScript. Cuando una función devuelve un valor, la instrucción que llamó a esa función, la tratará como si fuera una expresión.

Vayamos por pasos y veamos cómo crearlas, utilizarlas, etc.

Sintaxis

Te mostraré algunos ejemplos en un momento, pero, antes de nada, vamos a ver la sintaxis formal de una función:

```
function nombreFuncion ( [parametro1]....[parametroN] )
{
     // instrucciones
     [return valor]
}
```

Fíjate que si nuestra función va a **devolver algún valor** emplearemos la palabra reservada **return**, para hacerlo.

Para realizar una llamada a una función lo podemos hacer con:

```
nombreFuncion( );  // Esta llamada ejecutaría las instrucciones programadas dentro de la función.
```

Otro ejemplo de uso de una función que devuelve un valor y queremos recogerlo en una variable es utilizando una asignación:

```
variable = nombreFuncion( );  // En este caso la función devolvería un valor que se asigna a la variable.
```

Las funciones en JavaScript también son objetos, y como tal tienen métodos y propiedades. Un método, aplicable a cualquier función puede ser **toString()**, el cual nos devolverá el código fuente de esa función.

Parámetros

Cuando se realiza una llamada a una función, muchas veces es necesario pasar parámetros (también conocidos como argumentos). Este mecanismo nos va a permitir enviar datos entre instrucciones.

Para pasar parámetros a una función, tendremos que escribir dichos parámetros entre paréntesis y separados por comas.

A la hora de definir una función que recibe parámetros, lo que haremos es, escribir los nombres de las variables que recibirán esos parámetros entre los paréntesis de la función.

Veamos el siguiente ejemplo:

```
const saludar = (nombre) => {
      alert(`Hola ${nombre}`);
}
```

Si llamamos a esa función desde el código:

```
saludar('Bob Esponja');      //Mostraría una alerta con el texto: Hola Bob Esponja.
```

Los parámetros que usamos en la definición de la función *serán variables locales a la función*, y se inicializarán automáticamente en el momento de llamar a la función, con los valores que le pasemos en la llamada. En el siguiente apartado entraremos más en profundidad en lo que son las variables locales y globales.

Otro ejemplo de función que devuelve un valor podría ser el siguiente:

```
const calcularMayor = (num1, num2) => {
    return (num1 > num2) ? num1 : num2;
}
```

Esta función la podemos invocar lo mismo que la anterior, simplemente que debemos tener en cuenta que devuelve un valor:

```
console.log(calcularMayor(7, 5));
```

El número de argumentos en la llamada a una función no ha de coincidir necesariamente con el número de parámetros en la definición. Si se proporcionan argumentos «de más» se ignoran los sobrantes. Si son de menos, los otros toman el valor undefined.

Ámbito de las variables

Ha llegado la hora de distinguir entre las variables que se definen fuera de una función, y las que se definen dentro de las funciones.

Las variables que se definen fuera de las funciones se llaman **variables globales.** Las **variables** que se definen dentro de las funciones se llaman variables locales.

Una **variable global** en JavaScript tiene una connotación un poco diferente, comparando con otros lenguajes de programación. Para un script de JavaScript, el alcance de una variable global se limita al documento actual que está cargado en la ventana del navegador. Sin embargo, cuando inicializas una variable como variable global, quiere decir que todas las instrucciones de tu script (incluidas las instrucciones que están dentro de las funciones), tendrán acceso directo al valor de esa variable. Todas las instrucciones podrán leer y modificar el valor de esa variable global.

En el momento que una página se cierra, todas las variables definidas en esa página se eliminarán de la memoria para siempre. Si necesitas que el valor de una variable persista de una página a otra, tendrás que utilizar técnicas que te permitan almacenar esa variable.

En contraste a las variables globales, una **variable local será definida dentro de una función**. Antes viste que podemos definir variables en los parámetros de una función, pero también podrás definir nuevas variables dentro del código de la función.

El alcance de una variable local está solamente dentro del ámbito de la función. Ninguna otra función o instrucciones fuera de la función podrán acceder al valor de esa variable.

Reutilizar el nombre de una variable global como local es uno de los bugs más sutiles y por consiguiente más difíciles de encontrar en el código de JavaScript. La variable local en momentos puntuales ocultará el valor de la variable global, sin avisarnos de ello. Como recomendación, no reutilices un nombre de variable global como local en una función, y tampoco declares una variable global dentro de una función, ya que podrás crear fallos que te resultarán difíciles de solucionar.

Como ejemplo de ello puedes ver el siguiente ejemplo:

```
let nombre = 'Pepe';
const prueba = () => {
    let nombre = 'Pedro';
    console.log(nombre);      // Muestra por consola: Pedro
}
console.log(nombre);          // Muestra por consola: Pepe
prueba();
console.log(nombre);          // Muestra por consola: Pepe
```

Funciones anidadas

En JavaScript podemos anidar unas funciones dentro de otras. Es decir, podemos programar una función dentro de otra función.

Cuando no tenemos funciones anidadas, cada función que definamos será accesible por todo el código, es decir serán funciones globales. Con las funciones anidadas, podemos encapsular la accesibilidad de una función dentro de otra y hacer que esa función sea privada o local a la función principal. Tampoco te recomiendo el reutilizar nombres de funciones con esta técnica, para evitar problemas o confusiones posteriores.

Una buena opción para aplicar las funciones anidadas es cuando tenemos una secuencia de instrucciones que necesitan ser llamadas desde múltiples sitios dentro de una función, y esas instrucciones sólo tienen significado dentro del contexto de esa función principal. En otras palabras, en lugar de romper la secuencia de una función muy larga en varias funciones globales, haremos lo mismo pero, utilizando funciones locales.

Ejemplo de una función anidada:

```
const calcularHipotenusa = (cateto1, cateto2) => {
   const calcularCuadrado = (x) => { return x * x; }
   return Math.sqrt(calcularCuadrado(cateto1) + calcularCuadrado(cateto2));
}
console.log(calcularHipotenusa(1, 2)); // Muestra por consola: 2.23606797749979
```

Funciones predefinidas

Te mostraremos aquí una lista de funciones predefinidas, que se pueden utilizar a nivel global en cualquier parte de tu código de JavaScript. Estas funciones no están asociadas a ningún objeto en particular. Típicamente, estas funciones te permiten convertir datos de un tipo a otro tipo.

Funciones globales o predefinidas en JavaScript:

Funciones globales en JavaScript.

Función	Descripción
decodeURI()	Decodifica los caracteres especiales de una URL excepto: , / ? : @ & = + $ #
decodeURIComponent()	Decodifica todos los caracteres especiales de una URL.
encodeURI()	Codifica los caracteres especiales de una URL excepto: , / ? : @ & = + $ #
encodeURIComponent()	Codifica todos los caracteres especiales de una URL.
escape()	Codifica caracteres especiales en una cadena, excepto: * @ - _ + . /
eval()	Evalúa una cadena y la ejecuta si contiene código u operaciones.
isFinite()	Determina si un valor es un número finito válido.
isNaN()	Determina cuando un valor no es un número.
Number()	Convierte el valor de un objeto a un número.
parseFloat()	Convierte una cadena a un número real.
parseInt()	Convierte una cadena a un entero.
unescape()	Decodifica caracteres especiales en una cadena, excepto: * @ - _ + . /

1.2.- Uso avanzado de funciones

En JavaScript existen diversas formas declarar una función:

Declaración de función

Es la forma más tradicional y se utiliza la palabra clave function seguida del nombre de la función y un conjunto de paréntesis que pueden contener parámetros. Este tipo de funciones son "elevadas" (hoisted), lo que significa que se pueden llamar antes de su definición en el código.

```
function saludar() {
    console.log("Hola, mundo!");
}
```

Expresión de función

Aquí, la función se define como una expresión y se puede almacenar en una variable. Estas funciones no son elevadas, por lo que solo se pueden usar después de su definición.

```
const saludar = function() {
    console.log("Hola, mundo!");
};
```

Funciones anónimas

Las funciones anónimas son funciones sin nombre que se utilizan principalmente como argumentos a otras funciones. Pueden ser útiles en operaciones que se realizan una sola vez y no necesitan reutilizarse. Estas funciones no son elevadas, por lo que solo se pueden usar después de su definición.

```
setTimeout(function() {
    console.log("Hola, mundo!");
}, 1000);
```

Funciones flecha (arrow functions)

Las funciones flecha, o notación lamda, son una forma alternativa y más concisa de definir funciones, especialmente útiles para funciones pasadas como callbacks. Sin embargo, tienen algunas limitaciones en funcionalidad: no tienen su propio **this**, no se pueden utilizar con métodos como **bind**, **call** o **apply**, y no pueden ser usadas como constructores. Además, estas funciones no son elevadas, por lo que solo se pueden usar después de su definición.

```
// Función tradicional
function (a){
  return a + 100;
}

// Desglose de la función flecha

// 1. Elimina la palabra "function" y coloca la flecha entre el argumento y el corchete de apertura.
(a) => {
  return a + 100;
}

// 2. Quita los corchetes del cuerpo y la palabra "return" — el return está implícito.
(a) => a + 100;

// 3. Suprime los paréntesis de los argumentos
a => a + 100;
```

Además, existen múltiples funcionalidades que, aunque al principio pueden parecer complejas, facilitan el desarrollo en JavaScript:

Parámetros por defecto

Es también posible definir funciones con parámetros con valor por defecto (estos siempre tienen que ir al final). Esto es muy útil cuando una función de forma normal utiliza un valor y solo en casos concretos necesita otra. Por ejemplo, la función potencia puede usarse habitualmente para elevar un número al cuadrado, pero puntualmente puede querer usarse para otras potencias:

```
function potencia(a, b = 2) {
  return a ** b;
}
```

Esta función se podría utilizar de las siguientes formas:

```
potencia(2); // Devuelve 4
potencia(2,3); // Devuelve 8
```

Parámetros Rest

La sintaxis de los parámetros rest nos permiten representar un número indefinido de argumentos como un array.

```
function suma(...nums) {
  let total = 0;
  for (let n of nums) {
    total += n;
  }
  return total;
}

suma(1); // 1
suma(1,2); // 3
suma(1,2,3); // 6
```

También es posible acceder a todos los parámetros de una función tradicional con la variable arguments. arguments es un objeto similar a Array accesible dentro de funciones que contiene los valores de los argumentos pasados a esa función.

```
function func1(a, b, c) {
  console.log(arguments[0]);
  // Expected output: 1

  console.log(arguments[1]);
  // Expected output: 2

  console.log(arguments[2]);
  // Expected output: 3
}

func1(1, 2, 3);
```

Callback

Un callback (llamada de vuelta) es una función que recibe como argumento otra función y la ejecuta. Estas funciones son habitualmente conocidas como funciones de orden superior.

```
// Definimos una función que suma dos números y luego llama a un callback con el resultado
function sumar(a, b, callback) {
  const resultado = a + b;
  callback(resultado);
}

// Definimos una función que queremos usar como callback para mostrar el resultado
function mostrarResultado(resultado) {
  console.log("El resultado es: " + resultado);
}

// Llamamos a la función sumar con dos números y la función mostrarResultado como callback
sumar(5, 3, mostrarResultado);
```

Desestructuración

La desestructuración permite desempacar arrays o propiedades de objetos en distintas variables.

```
//Ejemplos sencillo
let [a,b] = [1,2];
console.log(a); // 1
console.log(b); // 2

//Ejemplo con rest
let [a,b,...rest] = [1,2,3,4];
console.log(a); // 1
console.log(b); // 2
console.log(rest); // [3,4]
```

Esto es útil a la hora de trabajar con funciones que queramos que devuelvan múltiples parámetros:

```
function divisionEntera(a,b){
    if(b==0) throw "No se puede dividir por 0";
    return [parseInt(a/b),a%b];
}

let [consciente,resto] = divisionEntera(5,2);
console.log(consciente); // 2
console.log(resto); // 1

divisionEntera(5,0); // Error: No se puede dividir por 0
```

O si se quiere hacer un for...of de un array multidimensional

```
let notas=[["Lengua",5],["Mates",7]];

for(let [asignatura,nota] of notas){
  console.log(`En ${asignatura} ha obtenido una calificación de ${nota}`);
}
//Salida:
//En Lengua ha obtenido una calificación de 5
//En Mates ha obtenido una calificación de 7
```

Otros tipos de funciones

Existen otros tipos de funciones que también son interesantes:

- Función asíncrona. Son funciones especiales en las que se puede esperar a que una operación termine (como una petición de datos al servidor) con el operador await. Profundizaremos más en ellas cuando trabajemos con asincronía.

```
async function peticionServidorFetch(){

  try {
    let response = await fetch('URL');
    let user = await response.json();
  } catch(err) {
    // atrapa errores tanto en fetch como en response.json
    alert(err);
  }
}
```

- Funciones generadoras. Los generadores son funciones de las que se puede salir y volver a entrar. Su contexto (asociación de variables) será conservado entre las reentradas. Se definen con **function*** y n lugar de utilizar return para salir, se utiliza yield.

```
function* foo(index) {
  while (index < 2) {
    yield index;
    index++;
  }
}

const iterator = foo(0);

console.log(iterator.next().value);
// Expected output: 0

console.log(iterator.next().value);
```

```
// Expected output: 1
```

2.- Estructuras de datos

Caso práctico



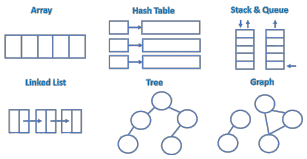
Ministerio de Educación (Uso educativo nc)

Antonio comienza a estudiar las estructuras de datos que le permitirán almacenar la información que gestionará en sus scripts, de una forma mucho más organizada. Existen muchos tipos de estructuras de datos, pero en este caso **Antonio** profundizará en una de ellas "los arrays".

Verá cómo crearlos, recorrerlos para consultar su información, borrar elementos del array, y las propiedades y métodos disponibles para su gestión.

Terminará estudiando lo que son arrays paralelos y arrays multidimensionales, los cuales le permitirán ampliar la capacidad y ventajas que aporta un array estándar.

Una estructura de datos es una forma particular de organizar datos que permite acceder y operar datos de manera eficiente. Existe una amplia variedad de estructuras de datos, desde las más simples, como los arrays, hasta las más complejas, como los grafos.



Jonathan Seow. *Estructuras de datos básicas: array, hash table (map/ diccionario), stack & queue (pila & cola), Linked List (lista enlazada), Tree (arbol), Graph (grafo).* (Todos los derechos reservados)

La elección de la estructura adecuada para cada problema específico es crucial, ya que cada una organiza la información de forma distinta, permitiendo realizar ciertas operaciones de manera más eficiente que otras. Si no seleccionamos la estructura de datos correcta, aunque la aplicación sea funcional, no será eficiente. Esto significa que operaciones como la eliminación de datos pueden llevar más tiempo del necesario, afectando el rendimiento general de la aplicación. Por tanto, comprender las características y ventajas de cada estructura de datos es esencial para optimizar el rendimiento y la eficacia del software.

Una estructura de datos se compone de un conjunto de valores de datos y las operaciones permitidas sobre esos datos. Por tanto, es fundamental comprender los datos con los que trabajamos y como se relacionan de cara a elegir la estructura de datos adecuada.

En esta unidad no hay tiempo suficiente para profundizar en todas las estructuras de datos, por lo que nos centraremos en las más utilizadas: arrays, Maps y Sets.

Autoevaluación

La elección de una estructura de datos adecuada no influye en el rendimiento general de una aplicación.

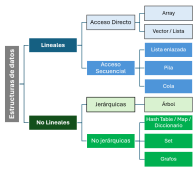
○ Verdadero ○ Falso

Falso

La elección de la estructura de datos adecuada es crucial, ya que cada una organiza la información de forma distinta y permite realizar ciertas operaciones de manera más eficiente que otras, afectando el rendimiento general de la aplicación.

2.1.- Principales estructuras de datos

Las estructuras de datos se organizan en lineales y no lineales.



Óscar Gómez. *Esquema de las estructuras de datos lineales y no lineales.* (Dominio público)

Lineales

Estructuras donde los elementos están organizados de manera secuencial, cada elemento tiene un único sucesor y, en algunos casos, un único predecesor. Dentro de estos pueden ser de acceso directo, es decir, se puede acceder directamente a cualquier valor; o secuenciales para acceder al valor en la posición n es necesario pasar por todas las posiciones anteriores.

Estructura	Tipo	Descripción	Ejemplos de uso
Array	Acceso directo	Colección de elementos del mismo tipo, almacenados en posiciones contiguas de memoria. Permiten acceso rápido a cualquier elemento mediante su índice, lo que facilita la búsqueda y actualización de datos. En JavaScript, los arrays son dinámicos, es decir, se redimensionan automáticamente, asemejándose más a lo que en otros lenguajes se conocen como listas o vectores.	✔ Listas de elementos ✔ Matrices
Vector / Lista	Acceso directo	Similar a los arrays, pero con la capacidad de redimensionarse automáticamente al añadir o eliminar elementos. Permiten acceso rápido y eficiente mediante índices. En muchos lenguajes, son una implementación específica de arrays dinámicos.	✔ Listas dinámicas de elementos, ✔ Almacenamiento de datos de gran tamaño
Lista Enlazada	Acceso secuencial	Colección de nodos donde cada nodo contiene un dato y una referencia al siguiente nodo. Son muy eficientes para inserciones y eliminaciones, ya que no requieren desplazamiento de elementos, pero el acceso a elementos es secuencial y puede ser más lento en comparación con los arrays. Existen variaciones como listas simplemente enlazadas, doblemente enlazadas y circularmente enlazadas.	✔ Lista de elementos de longitud variable ✔ Gestión de memoria dinámica
Pila	Acceso secuencial	Estructura LIFO (Last In, First Out). Los elementos se añaden (push) y se eliminan (pop) desde el mismo extremo, denominado "tope" de la pila.	✔ Gestión de llamadas de funciones ✔ Deshacer operaciones en editores de texto
Cola	Acceso secuencial	Estructura FIFO (First In, First Out). Los elementos se añaden (enqueue) al final de la cola y se eliminan (dequeue) del principio.	✔ Gestión de tareas en sistemas operativos ✔ Simulación de líneas de espera

No lineales

Estructuras donde los elementos no están organizados de manera secuencial, permitiendo mayor complejidad en las relaciones entre elementos. Se pueden distinguir entre jerárquicos, existe una relación de parentesco entre algunos elementos, y no jerárquicos no existe una jerarquía entre los elementos.

Estructura	Tipo	Descripción	Ejemplos de uso
Árbol	Jerárquicas	Estructura de datos compuesta por nodos, donde cada nodo contiene un valor y referencias (enlaces) a otros nodos (hijos). El nodo superior se llama raíz, y los nodos sin hijos se llaman hojas.	✔ Documento HTML. DOM.

		Los árboles pueden ser binarios, AVL, B-Trees, etc. Son eficientes para realizar búsquedas y mantener datos ordenados.	✓ Implementación de sistemas de archivos
Map (Diccionario o Hash Table)	No jerárquicas	Estructura que almacena pares de clave-valor. Se puede acceder directamente a cualquier valor a partir de su clave. Las implementaciones comunes incluyen tablas hash, que utilizan funciones hash para distribuir las claves uniformemente. Son eficientes para búsquedas, inserciones y eliminaciones.	✓ Datos de configuración ✓ Implementación de cachés
Set (Colección)	No jerárquicas	Colección de elementos únicos, donde no se permiten duplicados. Los sets proporcionan operaciones rápidas para verificar la presencia de un elemento, añadir nuevos elementos, y eliminar elementos existentes.	✓ Gestión de conjuntos de datos únicos ✓ Eliminación de duplicados
Grafo	No jerárquicas	Estructura de datos compuesta por nodos (vértices) y aristas (enlaces) que conectan pares de nodos. Los grafos pueden ser dirigidos (con direcciones en las aristas) o no dirigidos.	✓ Relaciones de usuarios en redes sociales ✓ Modelado de rutas y conexiones en mapas

Para saber más

La mayoría de las estructuras de datos no están implementadas por defecto en JavaScript, pero existen repositorios públicos donde podéis consultar implementaciones suyas:

https://github.com/TheAlgorithms/JavaScript

Puedes profundizar sobre estas estructuras en:

Las principales estructuras de datos que deberías saber para tu próxima entrevista de programación.

Road Map.

También puedes profundizar en inglés en:

7 estructuras de datos para JavaScript que debes conocer (inglés).

Estructuras de Datos por Google (inglés).

Estructuras de Datos por MIT (inglés).

2.2- Arrays

Un **array** es una de las mayores estructuras de datos proporcionadas para almacenar y manipular colecciones de datos. Un array se define como **una colección ordenada de datos a los que se puede acceder por posición** de forma aleatoria e impredecible. En JavaScript, los arrays son dinámicos y pueden crecer y decrecer de tamaño automáticamente, así como almacenar diferentes tipos de datos.

Los arrays nos permiten guardar un montón de elementos y acceder a ellos de manera independiente. Cada elemento es referenciado por la posición que ocupa dentro del array. Dichas posiciones se llaman **índices** y siempre son correlativos. En JavaScript cuando trabajamos con arrays se utiliza la **indexación base-cero:** en este modo, el primer elemento del array será la componente 0, es decir tendrá el índice 0.

```
//Declaramos un array
let diasSemana=["Lunes","Martes","Miercoles","Jueves","Viernes","Sabado","Domingo"];

//Acedemos a los valores del array. De manera aleatoria.
console.log(`El primer día de la semana es ${diasSemana[0]}` );
console.log(`El tercer día de la semana es ${diasSemana[2]}` );
console.log(`El último día de la semana es ${diasSemana[6]}` );
```

JavaScript emplea un montón de arrays internamente para gestionar los objetos HTML en el documento, propiedades del navegador, etc. En la unidad anterior si recuerdas, teníamos colecciones dentro del objeto **document**. Pues bien, cada una de esas colecciones (**anchors[], forms[], links[], e images[]**) será un array que contendrá las referencias de todas las anclas, formularios, enlaces e imágenes del documento.

Se puede considerar que todos los arrays son de una dimensión: la dimensión principal, pero los elementos de dicha fila pueden a su vez contener otros arrays o matrices, lo que nos permitiría hablar de **arrays multidimensionales** (los más fáciles de imaginar son los de una, dos y tres dimensiones). No es aconsejable utilizar arrays de más dimensiones, si nuestra lógica de programa no la requiere, ya que hacen difícil su comprensión.

Autoevaluación

En JavaScript los índices de los arrays comienzan en 1. ¿Verdadero o falso?

○ Verdadero ○ Falso

Falso

Los arrays en JavasScript comienzan con su índice en 0.

2.2.1.- Trabajando con Arrays



Everaldo Coelho (GNU/GPL)

Un array es una de las mayores estructuras de datos proporcionadas para almacenar y manipular colecciones de datos. A diferencia de otros lenguajes de programación, los arrays en JavaScript son muy versátiles, en el sentido de los diferentes tipos de datos que podemos almacenar en cada posición del array. En JavaScript un array puede contener elementos de diferentes tipos, lo que también los hace muy versátiles.

En programación, **un array se define como una colección ordenada de datos**. Lo mejor es que pienses en un array como si fuera una tabla que contiene datos, o también como si fuera una hoja de cálculo.

A medida que vayas diseñando tu aplicación, tendrás que identificar las pistas que te permitan utilizar arrays para almacenar datos. Por ejemplo, imagínate que tienes que almacenar un montón de coordenadas geográficas de una ruta a caballo: ésto si que sería un buen candidato para emplear una estructura de datos de tipo array, ya que podríamos asignar nombres a cada posición del array, realizar cálculos, ordenar los puntos, etc. Siempre que veas similitudes con un formato de tabla, será una buena opción para usar un array.

Creación

Para crear un array, podemos utilizar varias sintaxis. Por ejemplo, si tenemos las siguientes variables:

```
let coches = [ 'Seat', 'Audi', 'BMW', 'Toyota' ];
let numeros = [ 1, 5, 3, 9, 6, 4 ];
let diferentes = [ 'Pepe', 5, 'Juan', false ];
```

También es posible utilizar la sintaxis `new Array()` y luego ir asignado valores a los diferentes elementos como, por ejemplo:

```
let coches = new Array();
coches[0] = 'Seat';
coches[1] = 'Audi';
coches[2] = 'BMW';
coches[3] = 'Toyota';
```

En JavaScript también es posible utilizar arrays asociativos, es decir, asociaciones clave-valor. Veamos un ejemplo:

```
let edades = new Array();
edades['Juan'] = 20;
edades['Ana'] = 18;
edades['Pedro'] = 25;
console.log(edades);
console.log(edades['Juan']);
console.log(edades['Ana']);
console.log(edades['Pedro']);
```

Recorrido

Para recorrer un array podemos utilizar uno de los bucles que ya conocemos. Por ejemplo, dado el siguiente array:

```
let numeros = [ 1, 5, 3, 9, 6, 4 ];
```

Podríamos utilizar un bucle `for` para recorrerlo:

```
for (let i = 0; i < numeros.length; i++) {
    console.log(numeros[i]);
}
```

También existe un bucle especial para recorrer arrays en JavaScript; el denominado `for-in`:

```
for (let i in numeros) {
    console.log(numeros[i]);
}
```

Otro bucle que se puede utilizar es el `for-of`:

```
for (let n of numeros) {
    console.log(n);
}
```

Incluso podemos utilizar otras construcciones como la función `forEach`:

```
numeros.forEach(numero => console.log(numero));
```

Es más, podemos utilizar las funciones **map**, **reduce** y **filter** para realizar algunas acciones comunes con los arrays:

```
let cuadrados = numeros.map(numero => {return numero * numero});
console.log(cuadrados);
```

En el siguiente enlace se muestran ejemplos detallados de cómo utilizar estas nuevas características.

Uso de las funciones **map**, **filter** y **reduce**.

Trabajando con sus elementos

Una vez creado un array, siempre podremos añadir nuevos elementos de forma dinámica utilizando un indice no utilizado.

```
numeros[6] = 15;
console.log(numeros);
```

Debemos tener en cuenta que si no añadimos el nuevo elemento al final, las posiciones intermedias se quedarán con valores indefinidos.

Podemos eliminar un elemento del array mediante la función **delete**, pero esta función no reduce el tamaño del array, simplemente borra el elemento:

```
delete numeros[6];
console.log(numeros.length);    // Muestra por consola: 7
console.log(numeros[6]);        // Muestra por consola: undefined
```

Si lo que queremos es también reducir su tamaño deberemos utilizar la función **splice**; a esta función le indicamos la posición a partir de la cual empezamos a eliminar y el número de elementos:

```
numeros.splice(5, 2);
console.log(numeros);           // Muestra por consola: [ 1, 5, 3, 9, 6 ]
```

Propiedades y métodos

Propiedades del objeto array:

Propiedades del objeto Array

Propiedad	Descripción
constructor	Devuelve la función que creó el prototipo del objeto array
length	Ajusta o devuelve el número de elementos en un array
prototype	Te permite añadir propiedades y métodos a un objeto

Métodos del objeto array:

Métodos del objeto Array

Métodos	Descripción
concat()	Une dos o más arrays, y devuelve una copia de los arrays unidos
join()	Une todos los elementos de un array en una cadena de texto
pop()	Elimina el último elemento de un array y devuelve ese elemento
push()	Añade nuevos elementos al final de un array, y devuelve la nueva longitud
reverse()	Invierte el orden de los elementos en un array
shift()	Elimina el primer elemento de un array, y devuelve ese elemento
slice()	Selecciona una parte de un array y devuelve el nuevo array
sort()	Ordena los elementos de un array

Métodos	Descripción
splice()	Añade/elimina elementos a un array
toString()	Convierte un array a una cadena y devuelve el resultado
unshift()	Añade nuevos elementos al comienzo de un array, y devuelve la nueva longitud
valueOf()	Devuelve el valor primitivo de un array

Arrays paralelos

El usar arrays para almacenar información, aporta una gran versatilidad a las aplicaciones. Pero en algunos casos, podría ser muy útil hacer búsquedas en varios arrays a la vez, de tal forma que podamos almacenar diferentes tipos de información, en arrays que estén sincronizados.

Cuando tenemos dos o más arrays, que utilizan el mismo índice para referirse a términos homólogos, se denominan **arrays paralelos**.

Veamos el siguiente ejemplo:

```
let personajes = [ 'Bob Esponja', 'Calamardo', 'Patricio' ];
let color = [ 'amarillo', 'beige', 'rosa' ];
for (let i = 0; i < personajes.length; i++) {
    console.log(`${personajes[i]} es de color ${color[i]}`);
}
```

Arrays multidimensionales

Como ya sabemos, un array puede almacenar elementos de diferentes tipos, por lo que un elemento de un array, a su vez puede ser otro array y a eso es a lo que se conoce como arrays multidimensionales.

Veamos el mismo ejemplo anterior, pero con un array multidimensional.

```
let personajes = [
    [ 'Bob Esponja', 'amarillo'],
    [ 'Calamardo', 'beige' ],
    [ 'Patricio', 'rosa' ]
];
for (let i in personajes) {
    console.log(`${personajes[i][0]} es de color ${personajes[i][1]}`);
}
```

O si se quiere hacer un for...of de un array multidimensional se puede hacer uso de la desestrecutración de la siguiente forma:

```
let notas=[["Lengua",5],["Mates",7]];

for(let [asignatura,nota] of notas){
  console.log(`En ${asignatura} ha obtenido una calificación de ${nota}`);
}
//Salida:
//En Lengua ha obtenido una calificación de 5
//En Mates ha obtenido una calificación de 7
```

Autoevaluación

Un array en JavaScript siempre debe contener elementos del mismo tipo. ¿Verdadero o falso?

○ Verdadero ○ Falso

Falso
Como hemos visto, un array puede contener elementos de diferentes tipos.

Para saber más

Los arrays en JavaScript llevan funciones para ser trabajados como:

- Pila. Utilizando la función push() para insertar al final, y la función pop() para sacar el último insertado.
- Cola. Utilizando la función push() para insertar al final, la función shift() para sacar el primer elemento de la cola.

2.2.2.- Array functions

Los objetos de tipo `array` tienen una serie de métodos que permiten operar con todos, o parte, de los elementos de un array de forma rápida. Las array functions reciben por parámetro una callback función, habitualmente definidas con la nomenclatura de funciones flecha, y la aplican a los elementos del array. Estas funciones a menudo son conocidas como funciones de orden superior.

Existe una gran variedad de array functions, las más conocidas son:

Método	Descripción	Ejemplo

map	Devuelve un nuevo array con los resultados de la llamada a la función.	`//Función potencia`
		`function potencia(a) { return a**2}`
		
		`//Pruebo la función`
		`potencia(2); // 4`
		`potencia(3): // 9`
		
		`//Defino array`
		`let arr = [0,1,3,6];`
		`let arrPotencia = arr.map(potencia);`
		
		`//Muestro resultados`
		`console.log(arr); // [ 0, 1, 3, 6 ]`
		`console.log(arrPotencia); // [ 0, 1, 9, 36 ]`
		
		`//Ejemplo con función flecha para a^3`
		`console.log(arr.map(a => a**3 )); // [ 0, 1, 27, 216 ]`

every	Comprueba si todos los elementos del array cumplen o no la función pasada como callback.
	Si se quiere comprobar si algún elemento cumple, se puede utilizar la función some().
		`//Función es Par`
		`function esPar(a) { return a%2 == 0 }`
		
		`Pruebo la función`
		`console.log(1); //false`
		`console.log(2); //true`
		
		`//Defino array`
		`let arr1 = [0,1,3,6];`
		`let arr2 = [0,2,4];`
		
		`//Muestro resultados`
		`console.log(arr1.every(esPar)); // false`
		`console.log(arr.2every(esPar)); // true`

filter	Devuelve un array con todos los elementos que cumplan la condición implementada por la función dada.	`//Función es Mayor o Igual que 0`
		`const esMayorIgual0 = (a) => a>=0;`
		
		`//La pruebo`
		`esMayorIgual0(1); // true`
		`esMayorIgual0(0); // true`
		`esMayorIgual0(-1); // false`
		
		`//Defino array`
		`let arr1 = [0,-1,3,-6];`
		
		`//Filtro`
		`let arr2 = arr1.filter(esMayorIgual0)`
		`console.log(arr2); // [0,3]`
		
		`//Ejemplo con arrow function`
		`arr1.filter(a => a >=0 );`

find	Devuelve el valor del primer elemento del array que cumple la función de prueba proporcionada.	`const array1 = [5, 12, 8, 130, 44];`
		
		`const found = array1.find((element) => element > 10);`
		
		`console.log(found); // Salida esperada: 12`

reduce	Agrega todos los valores del array utilizando la función pasada como callback.	`//Defino un array numerotico, de notas de 0 a 10` `let arr = [5,7,3,5.6]`

		``
		//Obtengo la suma del array con la función reduce
		let sum = arr.reduce((a,b) => a+b);
		console.log(sum); // 20.6
		
		//Obtengo la media aritmética
		let avg = sum / arr.length
		console.log(avg); // 5.15
		``

sort	Devuelve los elementos del array ordenados. Por defecto, ordena de menor a mayor. Si se pasa una función, ordena en función al criterio definida por esta.	```
		//Ordenación por defecto (menor a mayor)
		[1,4,0].sort(); // [0, 1, 4]
		
		//Ordenación de mayor a menor
		[1,4,0].sort((a,b)=> b>a); //[4, 1, 0]
		
		//Ordenación de estructuras más complejas (objetos) de menor a mayor
		let arr1=[{nombre: "Juan", edad: 25},{nombre: "Ana", edad: 22}];
		arr1.sort((a,b) => a.edad > b.edad); // [{ nombre: "Ana", edad: 22 }, { nombre: "Juan", edad: 25 }]
		
		//Ordenación de estructuras más complejas (arrays multidimensionales) de menor a mayor
		let arr2=[["Juan",25],["Ana", 22]];
		arr2.sort((a,b) => a[1] > b[1]); // [["Ana", 22],["Juan",25]]
		``

Para saber más

Puedes profundizar sobre más array functions en:

- https://lenguajejs.com/javascript/arrays/array-functions/
- Ejemplos de uso de array functions (Twitter).

2.2.3.- Array vs NodeList

En JavaScript, un array y un nodeList son colecciones de elementos, pero tienen algunas diferencias clave en su funcionamiento y en cómo se manejan.

- **Array**. Un array es una estructura de datos que permite almacenar una colección de elementos. Los arrays son versátiles y cuentan con muchos métodos incorporados para manipular sus elementos.

```
let fruits = ['Apple', 'Banana', 'Cherry'];
console.log(fruits); // ["Apple", "Banana", "Cherry"]
```

- **NodeList**. Un nodeList es una colección de nodos (elementos del DOM) que se obtienen, por ejemplo, al usar métodos como document.querySelectorAll(). A diferencia de los arrays, **los nodeLists no tienen todos los métodos que los arrays tienen**.

```
// Seleccionar todos los elementos <p> en el documento
let paragraphs = document.querySelectorAll('p');
console.log(paragraphs); // NodeList(3) [p, p, p]
```

Podemos convertir un nodeList en un array utilizando Array.from(). Esto nos permite usar los array functions en una colección de nodos del DOM.

Ejemplo de conversión de NodeList a Array:

```
// Seleccionar todos los elementos <p> en el documento
let paragraphs = document.querySelectorAll('p');
console.log(paragraphs); // NodeList(3) [p, p, p]

// Convertir el NodeList a un Array
let paragraphsArray = Array.from(paragraphs);
console.log(paragraphsArray); // [p, p, p]

// Ahora podemos usar métodos de array
paragraphsArray.forEach(p => {
    p.style.color = 'blue'; // Cambiar el color de texto a azul
});
```

2.2.4.- Copia de arrays

Para trabajar con arrays y otras estructuras de datos, es crucial entender cómo funcionan las copias de datos, diferenciando entre copiar un valor (al trabajar con datos simples, p.ej. números) y copiar una referencia (p. ej. arrays u objetos).

Ejemplo de copia por valor

Primero, vamos a ver un ejemplo de copia por valor a la que ya estamos acostumbrados:

```
// Ejemplo de copia por valor con cadenas de texto
let str1 = "Hola";
let str2 = str1; // Aquí se copia el valor de 'str1' a 'str2'

console.log(str1); // "Hola"
console.log(str2); // "Hola"

str2 = "Adiós";

console.log(str1); // "Hola" (no cambia)
console.log(str2); // "Adiós"
```

Ejemplo de copia por referencia

Y ahora otro de copia por referencia:

```
let array1 = [1, 2, 3];
let array2 = array1; // Aquí se copia la referencia de 'array1' a 'array2'

console.log(array1); // [1, 2, 3]
console.log(array2); // [1, 2, 3]

array2.push(4);

console.log(array1); // [1, 2, 3, 4] (también cambia)
console.log(array2); // [1, 2, 3, 4]
```

Como puedes ver, al "copiar" por referencia al modificar el valor copiado también se modifica el original. Para evitarlo, se suele hacer uso del operador de desestructuración, por ejemplo:

Copia superficial de array

```
let array1 = [1, 2, 3];
let array2 = [...array1]; // shallow copy utilizando destructuración ... y luego reconstrucción

console.log(array1); // [1, 2, 3]
console.log(array2); // [1, 2, 3]

array2.push(4);

console.log(array1); // [1, 2, 3] (ahora no cambia)
console.log(array2); // [1, 2, 3, 4]
```

Copia superficial de objeto

```
let obj1 = { name: "Alice", age: 25 };
let obj2 = { ...obj1 }; // Copia superficial utilizando desestructuración

console.log(obj1); // { name: "Alice", age: 25 }
console.log(obj2); // { name: "Alice", age: 25 }

obj2.age = 30;
```

```
console.log(obj1); // { name: "Alice", age: 25 } (ahora no cambia)
console.log(obj2); // { name: "Alice", age: 30 }
```

Una vez visto este ejemplo, podemos distinguir entre los conceptos de copia superficial (shallow copy) y copia profunda (deep copy):

- **Copia superficial (shallow copy):** La copia superficial crea una nueva colección, pero no copia los elementos anidados dentro de la colección original. En su lugar, simplemente copia las referencias a los objetos anidados. Por lo tanto, si un elemento anidado se modifica en la copia, también se refleja en la colección original.
- **Copia profunda (deep copy):** La copia profunda, por otro lado, crea una nueva colección y recursivamente copia todos los objetos encontrados en la colección original, incluyendo cualquier elemento anidado. Esto significa que los cambios en la copia profunda no afectarán a la colección original, ya que los objetos anidados también han sido duplicados.

Recientemente, se ha añadido la función `structuredClone()` que permite realizar copias profundas con facilidad:

```
const mushrooms1 = {
  amanita: ["muscaria", "virosa"],
};

const mushrooms2 = structuredClone(mushrooms1);

mushrooms2.amanita.push("pantherina");
mushrooms1.amanita.pop();

console.log(mushrooms2.amanita); // ["muscaria", "virosa", "pantherina"]
console.log(mushrooms1.amanita); // ["muscaria"]
```

Para saber más

Puedes profundizar en:

- Cómo diferenciar entre copias superficiales y copias profundas en JavaScript (freecodecamp).

Autoevaluación

Cuando se realiza una copia por referencia de un objeto o array, cualquier modificación realizada en la copia también se reflejará en el objeto o array original.

○ Verdadero ○ Falso

Verdadero

En la copia por referencia, tanto la copia como el original apuntan al mismo espacio en memoria, por lo que los cambios en uno afectan al otro

2.3.- Map

El objeto `Map` almacena pares clave/valor. Tanto la clave como el valor pueden ser tanto tipo de datos primitivos como objetos.

Definición

Para definir un Map se utiliza `new Map()`.

```
//Definición de un Map vacío.
let miMapa = new Map();

//Definición de un Map con una serie de elementos
let miMapa2 =  new Map([
  [1, "uno"],
  [2, "dos"],
  [3, "tres"],
]);
```

Métodos

Método	Descripción	Ejemplo

set(key,value)	Añade o actualiza el valor de una clave.	`const map1 = new Map();`
		`map1.set('bar', 'foo');`
		
		`console.log(map1.get('bar'));`
		`// Expected output: "foo"`
		
		`console.log(map1.get('baz'));`
		`// Expected output: undefined`

get(key)	Devuelve el valor de una clave concreta.	

has(key)	Comprueba si el Map contiene una clave concreta.	`const map1 = new Map();`
		`map1.set('bar', 'foo');`
		
		`console.log(map1.has('bar'));`
		`// Expected output: true`
		
		`console.log(map1.has('baz'));`
		`// Expected output: false`

delete(key)	Elimina una clave y su valor asociado.	`const map1 = new Map();`
		`map1.set('bar', 'foo');`
		
		`console.log(map1.delete('bar'));`
		`// Expected result: true`
		`// True indicates successful removal`
		
		`console.log(map1.has('bar'));`
		`// Expected result: false`

size	Muestra el número de elementos del Map.	```const map1 = new Map();
		
		map1.set('bar', 'baz');
		map1.set(1, 'foo');
		
		console.log(map1.size);
		// Expected output: 2
		
		map1.clear();
		
		console.log(map1.size);
		// Expected output: 0```

clear()	Elimina todas las claves y sus valores asociados.	

Iteración

Si se quiere recorrer el Map se puede utilizar los métodos: keys() para recorrer todas las claves; values(), para los valores; y entries() para las parejas clave-valor.

```
let idAlumnoNotaMediaMap = new Map([
  ['oscar32', 6],
  ['alicia33', 7],
  ['julio47',    8]
]);

//Ejemplo de recorrer todas las claves
for (let k of idAlumnoNotaMediaMap.keys() ) { //equivalente a idAlumnoNotaMediaMap.entries()
  console.log(k); // oscar32 (y así consecutivamente)
}

//Ejemplo de recorrer todas las entradas
for (let v of idAlumnoNotaMediaMap.values() ) { //equivalente a idAlumnoNotaMediaMap.entries()
  console.log(v); // oscar32,6 (y así consecutivamente)
}

//Ejemplo de recorrer todos los valores
for (let entry of idAlumnoNotaMediaMap ) { //equivalente a idAlumnoNotaMediaMap.entries()
  console.log(entry); // 6 (y así consecutivamente)
}
```

Convertir en array/objeto

- Se puede convertir un Map a un array con **Array.from(map)**.
- Se puede convertir un Map a un objeto con **Object.fromEntries(map.entries())**.

Para saber más

Para **guardar** un Map (o un Set) en memoria local, localStorage o sessionStorage, no se puede utilizar directamente JSON.stringify, sino que es necesario convertirlo antes en un array.

```
// Crear un Map
const myMap = new Map();
myMap.set('key1', 'value1');
myMap.set('key2', 'value2');

// Convertir el Map a un objeto JSON
const jsonObject = JSON.stringify(Array.from(myMap.entries()));

// Guardar el objeto JSON en localStorage
localStorage.setItem('myMap', jsonObject);
```

Por otro lado, para cargar el mapa desde memoria local se debe:

```
// Obtener el objeto JSON desde localStorage
```

```
const jsonObject = localStorage.getItem('myMap');

// Parsear el JSON a un array de entradas
const entries = JSON.parse(jsonObject);

// Convertir el array de entradas a un Map
const myMap = new Map(entries);

console.log(myMap); // Map(2) {"key1" => "value1", "key2" => "value2"}
```

2.4.- Set

El objeto Set permite almacenar valores únicos de cualquier tipo, datos primitivos y objetos. No permite almacenar valores repetidos.

Definición

Para definir un Set se utiliza `new Set()`.

```
//Definición de un Set vacío.
let miSet = new Set();

//Definición de un Set con una serie de elementos
const miSet2 = new Set([1, 2, 3, 4, 5]);
```

Métodos

Método	Descripción	Ejemplo

add(value)	Añade un valor al Set	`const set1 = new Set();`
		
		`set1.add(42);`
		`set1.add(42);`
		`set1.add(13);`
		
		`for (const item of set1) {`
		`  console.log(item);`
		`  // Expected output: 42`
		`  // Expected output: 13`
		`}`

has(value)	Comprueba si el Set contiene un valor concreto	`const set1 = new Set([1, 2, 3, 4, 5]);`
		
		`console.log(set1.has(1));`
		`// Expected output: true`
		
		`console.log(set1.has(5));`
		`// Expected output: true`
		
		`console.log(set1.has(6));`
		`// Expected output: false`

delete(value)	Elimina un valor	`const mySet = new Set();`
		`mySet.add("foo");`
		
		`console.log(mySet.delete("bar")); // false; no se pudo borrar "bar" pues no estaba.`
		`console.log(mySet.delete("foo")); // true; borrado correctamente.`
		
		`console.log(mySet.has("foo")); // false; el valor ya no está en el Set.`

size	Muestra el número de elementos del Set	`const set1 = new Set();`
		`set1.add(1);`
		`set1.add('foo');`
		
		`console.log(set1.size);`
		`// Expected output: 2`
		
		`set1.clear();`
		
		`console.log(set1.size);`
		`// Expected output: 0`

clear()	Elimina todos los valores del Set	

Iteración

Si se quiere recorrer el Set se puede utilizar:

```
let set = new Set(["naranjas", "manzanas", "platanos"]);

for (let value of set) console.log(value);
```

Convertir en array

- Se puede convertir un Set a un array con **Array.from(set)**.

2.5.- WeakMap y WeakSet

WeakMap y WeakSet son versiones alternativas de Map y Set, respectivamente, en las que las claves solo pueden ser objetos. Esto significa que no se pueden utilizar datos primitivos, como cadenas o números, como claves. Son más eficientes en cuanto a la gestión de memoria, ya que las entradas son eliminadas automáticamente cuando el objeto original es eliminado. Sin embargo, no pueden realizar algunas operaciones como `clear()` o `size`.

WeakMap

```
>let estudiante = { name: "Ana" };

let estudianteNotaMap= new WeakMap();
weakMap.set(estudiante , 7);

john = null; // sobreescribe la referencia

// ¡Ana se eliminó de la memoria!
```

WeakSet

```
let votosSet= new WeakSet();

let oscar= { name: "oscar" };
let alicia= { name: "alicia" };

votosSet.add(oscar); // Oscar vota
votosSet.add(alicia); // luego Alicia
votosSet.add(oscar); // Oscar otra vez

// votosSet tiene 2 usuarios ahora

// comprobar si John nos visitó?
console.log(votosSet.has(oscar)); // true

// comprobar si Mary nos visitó?
console.log(votosSet.has(ana)); // false

oscar = null;

// visitedSet se limpiará automáticamente
```

Se utilizan como estructuras de datos "secundarias" además del almacenamiento de objetos "principal". Cuando se elimina el objeto principal, si solo se encuentra como clave en un WeakMap o dentro de un WeakSet, se limpiará automáticamente.

Autoevaluación

En un WeakMap, si un objeto utilizado como clave se elimina (por ejemplo, asignando su referencia a `null`), las entradas correspondientes en el WeakMap se eliminarán automáticamente de la memoria.

○ Verdadero ○ Falso

Verdadero

En un WeakMap, las claves deben ser objetos y si el objeto clave es eliminado, la entrada se limpia automáticamente de la memoria, lo que hace que WeakMap sea eficiente en la gestión de memoria.

Para saber más

Otro uso posible de estas estructuras es para relacionar nodos del DOM con objetos de datos (normalmente objetos JSON). Uno de los usos más habituales es vincular un objeto JSON recibido en una petición AJAX con un nodo del DOM.

Vamos a ver un ejemplo en el que en una web se generan aleatoriamente 5 elementos de una lista y a cada uno se le asocia un número secreto que se almacena en el WeakMap.

```
// Definimos las variables globales
let nodeToValueWeakMap;

// Inicialización de la página Web.
window.addEventListener("load", init);

function init() {
    nodeToValueWeakMap = new WeakMap();
    let lista = document.getElementById("lista");

    for (let i = 0; i < 5; i++) {
        // Creamos el nuevo nodo
        const newNodo = document.createElement("li");
        newNodo.textContent = `Elemento ${i}`;
        newNodo.addEventListener("click", showSecretNumber);
        lista.appendChild(newNodo);

        // Enlazamos el nodo a un número secreto
        const nSecreto = Math.floor(Math.random() * 100);
        nodeToValueWeakMap.set(newNodo, nSecreto);
    }
}

// Función para mostrar el número secreto
function showSecretNumber() {
    alert(`El número secreto de ${this.textContent} es ${nodeToValueWeakMap.get(this)}`);
}
```

2.6.- Rendimiento

Cuando elegimos una estructura de datos para un problema específico, es importante considerar cómo se desempeñará y qué operaciones necesitamos hacer con frecuencia. No todas las estructuras de datos y algoritmos funcionan igual de rápido para todas las tareas.

Por ejemplo, si queremos almacenar pares de idAlumno y nota, ¿qué es mejor usar: un array multidimensional o un Map? ¿Y cuál de estas opciones es más eficiente si necesitamos recorrer la estructura ordenada por nota, de menor a mayor?

Ejemplo Práctico

Vamos a probar esto con un ejemplo práctico usando JavaScript. Compararemos el rendimiento de un array multidimensional y un Map para almacenar y recorrer pares de **idAlumno** y **nota**. Usaremos `console.time()` y `console.timeEnd()` para medir el tiempo que toma cada operación.

Aquí tienes un código de ejemplo. Es algo complejo, pero con lo visto hasta ahora en esta unidad deberías ser capaz de comprenderlo:

Función `testArrayUserIDNote()` para probar arrays multimensionales

```
function testArrayUserIDNote(numberOfUsers){
  console.log("================================================");
  console.log(`===Pruebas con Array multidimensional - Tamaño ${numberOfUsers}===`);
  console.log("_____________________________________________________");

  console.time(`(Array-${numberOfUsers}) Creación de la estructura`);
  let array = [];
  for (let i = 0; i < numberOfUsers; i++) {
    array.push([i, Math.floor(Math.random() * 10)]);
  }
  console.timeEnd(`(Array-${numberOfUsers}) Creación de la estructura`);
  console.log("_____________________________________________________");


  console.log("(Array) BUSQUEDAS")
  let cases=[["Mejor",0],["Medio",parseInt(numberOfUsers/2)],["Peor",numberOfUsers-1],["Fuera",numberOfUsers*2]];

  for(let [text,searchedUserID] of cases){
    console.time(`(Array-${numberOfUsers}) Busqueda de un usuario (${text})`);
    let userArray = array.find(item => item[0] === searchedUserID);
    console.timeEnd(`(Array-${numberOfUsers}) Busqueda de un usuario (${text})`);
    //console.log(`(Array-${numberOfUsers}) Usuario (${searchedUserID} - ${text}) encontrado: ${userArray}`);
  }

  console.log("_____________________________________________________");

  console.time(`(Array-${numberOfUsers}) Ordenación de un usuario`);
  array.sort((a, b) => a[1] - b[1]); // Ordenar por nota
  console.timeEnd(`(Array-${numberOfUsers}) Ordenación de un usuario`);
  console.log("_____________________________________________________");
}
```

Función `testMapUserIDNote()` para probar Maps

```
function testMapUserIDNote(numberOfUsers){
  console.log("================================================");
  console.log(`===Pruebas con Map - Tamaño ${numberOfUsers}===`);
  console.log("_____________________________________________________");

  console.time(`(Map-${numberOfUsers}) Creación de la estructura`);
  let map = new Map();
  for (let i = 0; i < numberOfUsers; i++) {
    map.set(i, Math.floor(Math.random() * 10));
  }
  console.timeEnd(`(Map-${numberOfUsers}) Creación de la estructura`);

  console.log("_____________________________________________________");

  console.log("(Map) BUSQUEDAS")
  let cases=[["Mejor",0],["Medio",parseInt(numberOfUsers/2)],["Peor",numberOfUsers-1],["Fuera",numberOfUsers*2]];

  for(let [text,searchedUserID] of cases){
    console.time(`(Map-${numberOfUsers}) Busqueda de un usuario (${text})`);
    let userMap = map.get(searchedUserID);
    console.timeEnd(`(Map-${numberOfUsers}) Busqueda de un usuario (${text})`);
    //console.log(`(Map-${numberOfUsers}) Usuario (${searchedUserID} - ${text}) encontrado: ${userMap})`);
```

```
    }

    console.log("_____________________________________________________");

    console.time(`(Map-${numberOfUsers}) Ordenación de un usuario`);
    let sortedMap = new Map([...map.entries()].sort((a, b) => a[1] - b[1])); // Ordenar por nota
    console.timeEnd(`(Map-${numberOfUsers}) Ordenación de un usuario`);
    console.log("_____________________________________________________");
  }
```

Si lo probamos para el caso de una estructura de datos de tamaño 10.000, la salida por consola es:

```
// ===MAIN
testArrayUserIDNote(numberOfUsers=10000);
testMapUserIDNote(numberOfUsers=10000);
```

ARRAY	MAP
===Pruebas con Array multidimensional - Tamaño 10000=== _____ (Array-10000) Creación de la estructura: 2.911ms _____ (Array) BUSQUEDAS (Array-10000) Busqueda de un usuario (Mejor): 0.036ms (Array-10000) Busqueda de un usuario (Medio): 0.116ms (Array-10000) Busqueda de un usuario (Peor): 0.205ms (Array-10000) Busqueda de un usuario (Fuera): 0.327ms _____ (Array-10000) Ordenación de un usuario: 1.92ms _____	========== ===Pruebas con Map - Tamaño 10000=== _____ (Map-10000) Creación de la estructura: 1.878ms _____ (Map) BUSQUEDAS (Map-10000) Busqueda de un usuario (Mejor): 0.005ms (Map-10000) Busqueda de un usuario (Medio): 0.003ms (Map-10000) Busqueda de un usuario (Peor): 0.02ms (Map-10000) Busqueda de un usuario (Fuera): 0.003ms _____ (Map-10000) Ordenación de un usuario: 3.828ms _____

Si el objetivo es acceder a los datos rápidamente por clave (por ejemplo, buscar un idAlumno específico), es más adecuado utilizar la estructura de datos Map. Tiene un tiempo de búsqueda constante, se acceda al primer o al último elemento. Sin embargo, si necesitas ordenar los datos con frecuencia, un array puede ser más eficiente.

Estos tiempos pueden variar según la potencia de tu equipo. Centrarte en la comparativa entre estructuras más que en los valores de tiempo exactos.

Recomendación

Prueba el código visto en este apartado con varios tamaños de estructura de datos y hazte una tabla para confirmar la evolución del tiempo de búsqueda y ordenación. Prueba los siguientes casos: 1, 100, 10.000, 1.000.000 y 100.000.000. En los últimos casos, la función puede tardar un rato en ejecutarse.

Por ejemplo, con tamaño 10.000.000, los tiempos de búsqueda y ordenación pasan a ser:

ARRAY	MAP
_____ (Array) BUSQUEDAS (Array-10000000) Busqueda de un usuario (Mejor): 0.028ms (Array-10000000) Busqueda de un usuario (Medio): 45.422ms (Array-10000000) Busqueda de un usuario (Peor): 81.91ms (Array-10000000) Busqueda de un usuario (Fuera): 86.911ms _____ (Array-10000000) Ordenación de un usuario: 1.605s	_____ (Map) BUSQUEDAS (Map-10000000) Busqueda de un usuario (Mejor): 0.006ms (Map-10000000) Busqueda de un usuario (Medio): 0.004ms (Map-10000000) Busqueda de un usuario (Peor): 0.022ms (Map-10000000) Busqueda de un usuario (Fuera): 0.004ms _____ (Map-10000000) Ordenación de un usuario: 6.106s

Complejidad Big-O

Pero ¿cómo escala el tiempo según aumente el tamaño de la estructura de datos? Para responder a esta pregunta debemos entender el concepto de complejidad algoritmica, Big-O.

La complejidad Big-O nos dice cómo se comporta el tiempo de ejecución de un algoritmo a medida que aumenta el tamaño de los datos. Por ejemplo, ordenar

un array tiene una complejidad diferente dependiendo del algoritmo que usemos:

- O(1): Tiempo constante. La operación tarda lo mismo sin importar el tamaño del conjunto de datos.
- O(n): Significa que el tiempo de ejecución crece de manera proporcional al tamaño de los datos.
- O(n^2): Significa que, si duplicamos el tamaño de los datos, el tiempo de ejecución se cuadruplica.

El acceso de búsqueda en un Map es O(1), mientras que un array es de O(N). Por tanto, se confirma que es mejor utilizar el Map para casos donde tenemos que realizar muchas búsquedas en una estructura de datos.

Para saber más

Puedes profundizar más sobre complejidad y algoritmia en:

- Complejidad temporal (wikipedia).

3.- Clases y objetos

Caso práctico


Ministerio de Educación (Uso educativo nc)

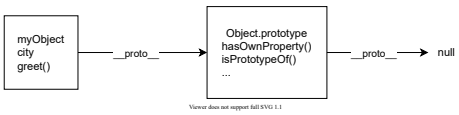
Antonio ha hecho una pausa para tomar café, y charla con **Juan** de todo lo que ha aprendido sobre las funciones. Comenta lo interesante que ha sido, pero le surge una duda, ya que cuando estudió los objetos en JavaScript, vio que tenían propiedades y métodos, y los métodos tienen el mismo formato que las funciones que acaba de estudiar. **Juan** le dice que efectivamente eso es así, porque en realidad los métodos son funciones que se asignan a un objeto determinado, y que a parte de los objetos que ya ha estudiado en unidades anteriores, en JavaScript podrá crear sus propios objetos, con los métodos (funciones) y propiedades que él quiera.

Terminan su café y los dos vuelven al despacho para buscar documentación sobre cómo crear nuevos objetos en JavaScript.

JavaScript es un **lenguaje basado en prototipos**, lo que implica ciertas peculiaridades al trabajar con clases y objetos en comparación con lenguajes basados en clases, como Java. En JavaScript, los objetos pueden tener un prototipo del cual heredan propiedades y métodos. Una característica distintiva de este enfoque es la posibilidad de instanciar objetos sin necesidad de una clase, así como la capacidad de añadirles propiedades y métodos en tiempo de ejecución.

```
const pos1 = {
    x: 0,
    y: 0,
    toString() {
        console.log(`Punto en: (${this.x}, ${this.y})`)
    }
};
```

Un concepto fundamental en JavaScript es la cadena de prototipos. Cuando se intenta acceder a una propiedad o método de un objeto, JavaScript busca primero en el propio objeto. Si no lo encuentra, continúa buscando en el prototipo del objeto, y así sucesivamente, hasta llegar al final de la cadena de prototipos.



Mozilla.org. *Cadena de prototipos en JavaScript.* (CC BY-SA)

Originalmente, el trabajo con clases y objetos en JavaScript se realizaba utilizando funciones y aprovechando el encapsulamiento. Sin embargo, con la evolución de ECMAScript, se han introducido nuevas funcionalidades que permiten trabajar con clases y objetos de una manera más similar a otros lenguajes de programación orientados a objetos,sin perder la flexibilidad y poder del sistema basado en prototipos.

JavaScript moderno (con class)

```
class Persona {
    constructor(nombre) {
        this.nombre = nombre;
    }

    saludar() {
        console.log(`Hola, mi nombre es ${this.nombre}`);
    }
}

const persona1 = new Persona('Juan');
persona1.saludar(); // Hola, mi nombre es Juan
```

JavaScript clásico (con funciones y encapsulamiento)

```
function Persona(nombre) {
    this.nombre = nombre;

    this.saludar = function() {
        console.log('Hola, mi nombre es ' + this.nombre);
    }
}

var persona1 = new Persona('Juan');
persona1.saludar(); // Hola, mi nombre es Juan
```

En esta sección, exploraremos tanto la forma de trabajar con objetos instanciados sin ninguna clase, como el uso de clases en el JavaScript moderno.

Para saber más

Puedes profundizar sobre la programación basada en prototipos en:

- https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/Object_prototypes
- JavaScript: POO — Programación basada en prototipos

También puedes profundizar sobre el funcionamiento del recolector de basura en Javascript en:

- https://es.javascript.info/garbage-collection

3.1.- Objetos

Un objeto en JavaScript es realmente una colección de propiedades. Las propiedades pueden tener forma de datos, tipos, funciones (métodos) o incluso otros objetos. De hecho, sería más fácil de entender un objeto como un array de valores, cada uno de los cuales está asociado a una propiedad (un tipo de datos, método u objeto). Un momento: ¿un método puede ser una propiedad de un objeto? Pues en JavaScript parece que sí.

Una función contenida en un objeto se conoce como un método. Los métodos no son diferentes de las funciones que has visto anteriormente, excepto que han sido diseñados para ser utilizados en el contexto de un objeto, y por lo tanto, tendrán acceso a las propiedades de ese objeto. Esta conexión entre propiedades y métodos es uno de los ejes centrales de la orientación a objetos.

```
const x = {
    nombre: "Alicia María",
    apellidos: "Gómez Casado",
    edad: 18
};
```

El literal {} representa un objeto vacío (sin atributos)

```
const y = {};
```

A breve resumen de su síntesis:

Acceso

Se accede a las propiedades con el operador . o mediante el operador corcherte [].

```
x.nombre; // Alicia María
x["nombre"]; // Alicia María

x.noexiste; // undefined
```

Además, JavaScript tiene la peculiaridad que es posible añadir atributos a objetos ya creados:

```
x.curso="DAW";
```

Además, el operador in permite determinar la existencia de un atributo dentro de un objecto:

```
if ("edad" in x) {
    console.log("x tiene un atributo llamado 'edad'");
}
```

c

Autoevaluación

¿Cuál de las siguientes opciones permite prevenir la adición de nuevas propiedades a un objeto y la eliminación de propiedades existentes, pero permite modificar el valor de las propiedades existentes?

- ○ Object.freeze()
- ○ Object.seal()
- ○ Object.preventExtensions()
- ○ Object.assign()

Incorrecto. No permite modificar el valor de las propiedades existentes.

Opción correcta

Incorrecto. Permite eliminar propiedades existentes.

Incorrecto. Object.assign() copia todas las propiedades enumerables de uno o más objetos fuente a un objeto destino. Devuelve el objeto destino.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

Para saber más

¿Qué sucede con los métodos de un objeto si lo conviertes a JSON con `JSON.stringify()` y lo conviertes de nuevo a objeto con `JSON.parse()`?

3.2.- Clases



Everaldo Coelho (GNU/GPL)

Las clases en programación son representaciones de entidades del mundo real, donde se definen propiedades (atributos) y métodos (comportamientos). Esta clase es posteriormente instanciada en objetos, que ya tienen valores concretos y que son instancias específicas de esa clase, permitiendo manipular y gestionar sus datos.

Las clases proporcionan una herramienta de organización del código y de abstracción fundamental en la programación moderna, tales como la herencia o el polimorfismo. En JavaScript, como ya se comentó previamente, se trabaja con prototipos, aunque desde ECMAScript 6 se introdujeron mejoras sintácticas (como la palabra reservada **class**) que permiten utilizar clases de una manera más intuitiva y familiar para los desarrolladores acostumbrados a la programación orientada a objetos.

A continuación, puedes ver un resumen de los puntos clave del trabajo con clases en JavaScript:

Definición

Para definir una clase, que será el molde que nos permitirá crear objetos de dicha clase, simplemente debemos utilizar la siguiente sintaxis:

```
class NombreClase {
      // Cuerpo de la clase
}
```

Para nombrar una clase comenzamos su identificador con la primera letra en mayúsculas.

Por ejemplo, pensemos que queremos gestionar una serie de contactos, para ello podríamos empezar por definir una clase para un **Contacto** genérico:

```
class Contacto {
      // Cuerpo de nuestra clase Contacto
}
```

Creación

Para crear objeto de una clase dada simplemente debemos utilizar el operador **new** seguido del nombre de la clase.

```
let pepe = new Contacto();
let juan = new Contacto();
```

En breve veremos qué estamos haciendo al realizar esta operación, pero por ahora nos quedamos con que de esta forma podemos crear nuevos objetos de dicha clase.

Propiedades

Pero una clase no tendría sentido sin tener unas propiedades que sean específicas de cada uno de los objetos. A eso se le suele llamar estado de un objeto, ya que las propiedades de un objeto dado serán diferentes a las de otro.

Para declarar propiedades dentro de una clase simplemente las podemos declarar en el cuerpo de la clase, aunque como veremos ahora mismo, lo ideal es hacerlo en el método constructor. Para referirnos a las propiedades debemos anteponer la palabra reservada **this** al nombre de la propiedad como veremos en breve.

```
class Contacto {
    nombre;
    correo;
}
```

Ahora podremos crear objetos de dicha clase e inspeccionar sus propiedades.

```
let juan = new Contacto();
console.log(juan);      // Muestra por consola: Object { nombre: undefined, correo: undefined }
```

Podemos acceder a las propiedades de un objeto utilizando el operador .

```
let juan = new Contacto();
console.log(juan.nombre);      // Muestra por consola: undefined
juan.nombre = 'Juan';
console.log(juan.nombre);      // Muestra por consola: Juan
```

Constructor

Como hemos comentado anteriormente, lo ideal es crear y asignar valores a las propiedades en el método constructor. El método constructor es una función especial que es llamada cuando utilizamos el operador **new** de dicha clase. Dicho método, si no es definido, es definido por defecto, pero lo ideal es que nosotros lo definamos y le demos la funcionalidad adecuada, que suele ser inicializar las propiedades.

```
class Contacto {
    constructor(nombre, correo) {
        this.nombre = nombre;
        this.correo = correo;
    }
}
```

En el siguiente ejemplo creamos dos objetos de la clase: en el primero utilizamos el constructor definido por nosotros mismos y en el segundo el constructor definido por defecto.

```
let pepe = new Contacto('Pepe', 'pepe@gmail.com');
console.log(pepe);      // Muestra por consola: Object { nombre: "Pepe", correo: "pepe@gmail.com" }
let juan = new Contacto();
console.log(juan);      // Muestra por consola: Object { nombre: undefined, correo: undefined }
```

JavaScript no permite múltiples constructores para una misma clase, por lo que si se quiere simular este comportamiento es necesario comprobar el valor de las propiedades pasadas al constructor y ejecutar unas instrucciones u otras dependiendo de la casuística:

```
 constructor(newName){
        //Comprobar si se ha pasado el parametro newName
        if (typeof newName === 'undefined') { //si no se ha pasado
            // Ejecutar constructor vacio, por ejemplo, generar nombre aleatorio
            this.name="Fulanito";
        }
        else{
            this.name=newName;
        }
    }
```

Aunque en este caso concreto, al ser bastante simple, el código se podría simplificar haciendo uso del operador **??**:

```
constructor(newName){
        this.name=newName ?? "Fulanito";
}
```

Métodos

Los métodos son funciones definidas en la propia clase que determinan el comportamiento de los objetos creados.

Por ejemplo, podemos crear un método para nuestra clase encargado de devolver un mensaje de saludo que incluya los datos del objeto.

```
class Contacto {

    nombre;
    correo;

    constructor(nombre, correo) {
        this.nombre = nombre;
        this.correo = correo;
    }

    saludar() {
        return `Hola, soy ${this.nombre} y mi correo es ${this.correo}`;
    }
}

let pepe = new Contacto('Pepe', 'pepe@gmail.com');
console.log(pepe.saludar());   // Muestra por consola: Hola soy Pepe y mi correo es pepe@gmail.com
```

Propiedades y métodos estáticos

A veces se quiere poder acceder a propiedades o métodos de una clase sin la necesidad de instanciarla en un objeto. Por ejemplo, podríamos necesitar una función que convierta unidades de medida o que valide ciertos datos comunes a todos los objetos de esa clase. Esto se realiza utilizando propiedades y métodos estáticos y para declararlos debemos anteponer la palabra reservada **static** al nombre del método.

Vamos a ver un ejemplo de un conversar entre unidades de temperatura:

```
class Conversor {
    static celsiusAFahrenheit(celsius) {
        return (celsius * 9/5) + 32;
    }

    static fahrenheitACelsius(fahrenheit) {
        return (fahrenheit - 32) * 5/9;
    }

    static metrosAKilometros(metros) {
        return metros / 1000;
    }

    static kilometrosAMetros(kilometros) {
        return kilometros * 1000;
    }
}

// Uso de métodos estáticos
console.log(Conversor.celsiusAFahrenheit(30)); // Output: 86
console.log(Conversor.fahrenheitACelsius(86)); // Output: 30
console.log(Conversor.metrosAKilometros(5000)); // Output: 5
console.log(Conversor.kilometrosAMetros(5)); // Output: 5000
```

Modificadores de acceso

En ECMAScript 2020 se ha añadido la posibilidad de definir propiedades y métodos privados utilizando `#`.

```
class PruebaAcceso {
  #numeroSecreto = 0;
}

let p = new PruebaAcceso();
p.#numeroSecreto;  //Genera error.  Uncaught SyntaxError: reference to undeclared private field or method #numeroSecreto
```

Get y Set

Seguramente estés ya acostumbrado a acceder a propiedades privadas, como Nombre, utilizando métodos como **setNombre(nuevoNombre)** y **getNombre()**. Estos son habitualmente conocidos como getters y setters y permiten controlar que las propiedades no lleguen a tener valores incorrectos, por ejemplo, asignar a una variable edad una cadena.

En JavaScript, además se añaden las palabras reservadas get y set. Estas permiten definir métodos que se comportan como propiedades, proporcionando una sintaxis más limpia y fácil de usar para acceder y modificar valores de propiedades privadas.

Con Getters/Setters Clásicos

```
class Persona {
    #nombre;
    #edad;

    constructor(nombre, edad) {
        this.#nombre = nombre;
        this.#edad = edad;
    }

    setNombre(nuevoNombre) {
        this.#nombre = nuevoNombre;
    }

    getNombre() {
        return this.#nombre;
    }

    setEdad(nuevaEdad) {
        if (typeof nuevaEdad === 'number') {
            this.#edad = nuevaEdad;
        } else {
            console.error('La edad debe ser un número.');
        }
    }

    getEdad() {
        return this.#edad;
    }
}

const persona1 = new Persona('Juan', 30);
persona1.setNombre('Carlos');
console.log(persona1.getNombre()); // Output: Carlos
persona1.setEdad('treinta'); // Output: La edad debe ser un número.
```

Con las Palabras Reservadas get y set

```
class Persona {
    #nombre;
    #edad;

    constructor(nombre, edad) {
        this.#nombre = nombre;
        this.#edad = edad;
    }

    set nombre(nuevoNombre) {
        this.#nombre = nuevoNombre;
    }

    get nombre() {
        return this.#nombre;
    }

    set edad(nuevaEdad) {
        if (typeof nuevaEdad === 'number') {
            this.#edad = nuevaEdad;
        } else {
            console.error('La edad debe ser un número.');
        }
    }

    get edad() {
        return this.#edad;
    }
}

const persona2 = new Persona('Ana', 25);
persona2.nombre = 'María';
console.log(persona2.nombre); // Output: María
persona2.edad = 'veinticinco'; // Output: La edad debe ser un número.
```

Esto también se puede utilizar para variables autocalculadas, como el area de un cuadrado que es calculada a partir de sus lados

```
class Cuadrado {
    #lado;

    constructor(lado) {
        this.#lado = lado;
    }

    set lado(nuevoLado) {
        this.#lado = nuevoLado;
    }

    get lado() {
        return this.#lado;
    }

    get area() {
        return this.#lado * this.#lado;
    }
}

const cuadrado = new Cuadrado(4);
console.log(cuadrado.area); // Output: 16
cuadrado.lado = 5;
console.log(cuadrado.area); // Output: 25
```

Herencia

En JavaScript también podemos crear jerarquías de clases mediante la herencia. Una clase puede heredar de otra previamente declarado y por lo tanto heredará su estructura y su comportamiento. Para que una clase herede de otra se utiliza la palabra reservada extends y para llamar acceder funciones del padre se utilizar, super().

Por ejemplo, podríamos tener una clase para almacenar contactos telefónicos en la que queremos almacenar además del nombre y el correo, el número de teléfono. También queremos que el método saludar también devuelva información del teléfono.

```
class ContactoTelefonico extends Contacto {

    telefono;

    constructor(nombre, correo, telefono) {
        super(nombre, correo);
        this.telefono = telefono;
    }

    saludar() {
        return `${super.saludar()} y mi teléfono es ${this.telefono}`;
    }
}

let maria = new ContactoTelefonico('María', 'maria@gmail.com', '666112233');
console.log(maria.saludar());    // Muestra por consola: Hola soy María y mi correo es maria@gmail.com y mi teléfono es 666112233
```

Polimorfismo (avanzado)

Además, el uso de herencias permite hacer uso también de polimorfismo. El polimorfismo es una técnica de programación, algo más avanzada, que permite sacarle todo el potencial a la abstracción de clases y herencia. El polimorfismo permite que una misma interfaz pueda ser utilizada para distintos tipos de objetos, lo que facilita la reutilización de código y la extensión de funcionalidades sin modificar el código existente.

Imagina que queremos tener diferentes tipos de contactos: **ContactoTelefonico** y **ContactoPostal**. Ambos comparten la estructura básica de **Contacto**, pero tienen implementaciones específicas de su método saludar.

```
class Contacto {
    constructor(nombre, correo) {
        this.nombre = nombre;
        this.correo = correo;
    }

    saludar() {
        return `Hola, soy ${this.nombre} y mi correo es ${this.correo}`;
    }
}

class ContactoTelefonico extends Contacto {
    constructor(nombre, correo, telefono) {
        super(nombre, correo);
        this.telefono = telefono;
    }

    saludar() {
        return `${super.saludar()} y mi teléfono es ${this.telefono}`;
    }
}

class ContactoPostal extends Contacto {
    constructor(nombre, correo, direccion) {
        super(nombre, correo);
        this.direccion = direccion;
    }

    saludar() {
        return `${super.saludar()} y mi dirección es ${this.direccion}`;
    }
}

let contactos = [
    new ContactoTelefonico('María', 'maria@gmail.com', '666112233'),
    new ContactoPostal('Carlos', 'carlos@gmail.com', 'Calle Falsa 123')
];

contactos.forEach(contacto => {
    console.log(contacto.saludar());
});
```

En este ejemplo, hemos definido dos clases que heredan de Contacto: ContactoTelefonico y ContactoPostal. Ambas clases sobrescriben el método saludar para añadir información específica. Utilizando polimorfismo, podemos almacenar objetos de ambos tipos en un mismo array y llamar al método saludar sin preocuparnos de qué tipo de contacto es cada objeto. El método correcto será llamado automáticamente dependiendo del tipo de objeto.

Este enfoque permite extender fácilmente el programa añadiendo nuevas clases de contactos sin modificar el código que utiliza estas clases, favoreciendo la mantenibilidad y escalabilidad del código.

Para saber más

Puedes seguir profundizando sobre el uso de clases en JavaScript en las siguientes webs:

- https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/Classes_in_JavaScript
- https://es.javascript.info/classes

Seguramente recordaras que en JavaScript por defecto no se pueden utilizar enumeradores. Sin embargo, haciendo uso de objetos o de clases es posible saltarse esta limitación como puedes profundizar en el siguiente enlace:

- 4 formas de crear un enumerador en JavaScript (inglés).

3.2.1.- Uso de this en clases y su perdida

La palabra clave **this** hace referencia a diferentes objetos dependiendo del contexto de la función:

Contexto global

En el contexto global (fuera de cualquier función u objeto), **this** se refiere al objeto global **window**.

```
console.log(this); // En un navegador, esto imprimirá el objeto window.
```

En un objeto

Cuando this se usa dentro de un método de un objeto, se refiere al objeto en sí.

```
const persona = {
  nombre: "Juan",
  saludar: function() {
    console.log("Hola, " + this.nombre);
  }
};

persona.saludar(); // "Hola, Juan"
```

Manejador de eventos

En los manejadores de eventos, **this** se refiere al elemento del DOM que desencadenó el evento.

```
<button id="miBoton">Haz clic</button>

<script>
document.getElementById("miBoton").onclick = function() {
  console.log(this); // Apuntará al botón que fue clicado
};
</script>
```

Un problema común en JavaScript es **la pérdida del contexto de this** cuando una función de un objeto es pasada como callback o manejador de eventos.

Callback

```
const persona = {
  nombre: "Juan",
  saludar: function() {
    console.log("Hola, " + this.nombre);
  }
};

const saludar = persona.saludar;
saludar(); // Esto imprimirá "Hola, undefined" porque this no se refiere a persona.
```

Manejador de eventos

```
const persona = {
      nombre: "Juan",
      saludar: function() {
        console.log("Hola, " + this.nombre);
```

```
    }
};

// Intentando asignar el método saludar directamente como manejador de eventos
document.getElementById("miBoton").onclick = persona.saludar;

// Esto imprimirá "Hola, undefined" cuando se haga clic en el botón,
// porque `this` dentro de `saludar` no se refiere al objeto `persona`.
```

Para solucionar este problema, podemos usar el método **bind()** para asegurar que **this** dentro del método hace referencia al objeto.

Callback

```
const saludar = persona.saludar.bind(persona);
saludar(); // "Hola, Juan"
```

Manejador de eventos

```
document.getElementById("miBoton").onclick = persona.saludar.bind(persona);
```

Para saber más

Aparte de utilizando **bind()**, También se puede conservar el valor de **this** utilizando **call()** o **apply()**, si se quiere llamar directamente a la función con un valor concreto de **this**. Puedes profundizar más sobre sus diferencias en el siguiente enlace:

Cómo usar los métodos Call, Apply y Bind en javascript

3.3.- Modulos

En un principio los programas JavaScript eran bastante sencillos y podían almacenarse con facilidad en un único fichero. Sin embargo, con el surgimiento y evolución de las aplicaciones Webs estos se han ido complicando y requiriendo un mayor trabajo de organización y abstracción. En ECMAScript 6 se introducen los módulos que permiten la importación y exportación de código entre diferentes ficheros JavaScript, permitiendo trabajar de forma más flexible desde el código Javascript. Por ejemplo, almacenando cada clase en un archivo diferente o almacenando la configuración o constantes en un archivo independiente.

Puesta en marcha

Para trabajar con módulos es obligatorio importar desde el archivo HTML nuestro fichero JavaScript principal con la propiedad `type="module"`.

```
<script type="module" src='main.js'></script>
```

Si no se obtendrá el siguiente error:

```
Uncaught SyntaxError: Cannot use import statement outside a module
```

Exportar

Para exportar una **propiedad, función o clase** desde un archivo independiente se utiliza la palabra reservada export. Los archivos de módulos deben nombrarse con la extensión `.mjs` o `.js`, aunque se recomienda utilizar la extensión `.mjs` para indicar claramente que se está trabajando con un módulo y ayuda a que sean identificados como tal por herramientas como Node.js o Babel.

Es posible realizar la exportación antes de la declaración:

```
export const idioma = "es";
```

O de forma posterior:

```
const idioma = "es";

export {idioma};
```

Autoevaluación

Para trabajar con módulos en JavaScript, es obligatorio importar el archivo JavaScript principal desde el archivo HTML utilizando la propiedad `type="module"`.

○ Verdadero ○ Falso

Verdadero

Verdadero. Es necesario especificar `type="module"` en el script para habilitar la funcionalidad de módulos en JavaScript, de lo contrario se producirá un error.

Importar (estático)

De cara a importar los módulos en otros archivos, se utiliza la palabra reservada import. Se puede importar de forma selectiva:

```
import { idioma } from "./config.mjs";

console.log(idioma);   // es
```

O se puede importar todo el módulo:

```
import * as Config from './config.mjs'

console.log(Config.idioma); // es
```

Se deben utilizar siempre al principio del archivo.

Importar (dinámico)

Además, también es imposible importar un módulo solamente dentro de un contexto, es decir, solo donde es necesario con import().

```
import("/config.mjs").then((Config) => {
  console.log(Config.idioma); // es
});
```

Debes conocer

Si se quiere exportar una única cosa por módulo se puede utilizar **export default**:

```
export default idioma;
```

De esta forma se puede realizar la importación sin los corchetes {}:

```
import idioma from './config.mjs'; // no {idioma}

console.log(idioma); // es
```

Cuando se trabaja con clases es muy práctico permitiendo exportar la clase solo con:

```
export default class User {
  constructor(name) {
    this.name = name;
  }
}
```

E importarla con:

```
import User from './user.mjs';
```

Para saber más

Puedes profundizar sobre el uso de módulos en: https://es.javascript.info/import-export

A veces hay confusión entre la palabra reservada import, utilizada para el trabajo con módulos en el navegador Web, y la palabra reservada requiere, utilizada para la gestión de dependencias con Node.js. Puedes profundizar más sobre esto en: https://keepcoding.io/blog/import-y-require-en-javascript/

4.- Patrones de diseño

Caso práctico



Ministerio de Educación (Uso educativo nc)

Bajo la tutoría de **Ana**, **Antonio** se dispone a conocer las bases de los patrones de diseño más importantes en desarrollo de software.

Antonio ha escuchado a sus compañeros hablar sobre la importancia de estos patrones, pero aún no tiene un conocimiento claro de qué son y cómo aplicarlos. Ana, con su experiencia, le guiará a través de los conceptos clave y le enseñará a reconocer y utilizar algunos de los patrones más comunes en sus proyectos.

Un patrón de diseño (*design pattern*) es una solución habitual a un problema típico en el diseño de software. Los patrones de diseño se pueden categorizar según el tipo de problema que abordan en tres grandes grupos:

Patrones Creacionales	Patrones Estructurales	Patrones de Comportamiento
✓ Se enfocan en los mecanismos de creación de objetos ✓ Buscan desacoplar la creación de objetos de su uso ✓ Mejoran la flexibilidad y reutilización del código	✓ Se centran en la composición de clases y objetos ✓ Facilitan el diseño de estructuras complejas ✓ Combinan objetos y clases sin alterar su estructura básica	✓ Abordan las interacciones y comunicaciones entre clases y objetos ✓ Mejoran la asignación de responsabilidades y el flujo de control ✓ Facilitan una mejor colaboración entre los objetos

Los patrones de diseño más habituales utilizados en JavaScript son:

Patrón	Tipo	Descripción y propósito	Explicación detallada en:	Código JavaScript en:
Singleton	Creacional	El patrón Singleton asegura que una clase tenga una única instancia y proporciona un punto de acceso global a ella	https://refactoring.guru/es/design-patterns/singleton	https://www.patterns.dev/vanilla/singleton-pattern
Factory	Creacional	El patrón Factory define una interfaz para crear objetos, pero permite a las subclases alterar el tipo de objetos que se crearán.	https://refactoring.guru/es/design-patterns/factory-method	https://www.patterns.dev/vanilla/factory-pattern
Builder	Creacional	El patrón Builder separa la construcción de un objeto complejo de su representación, permitiendo que el mismo proceso de construcción cree diferentes representaciones.	https://refactoring.guru/es/design-patterns/builder	https://medium.com/@jesusmurfontanals/builder-pattern-con-javascript-1ee3f9e2b7e9
Adapter	Estructural	El patrón Adapter permite que clases con interfaces incompatibles trabajen juntas, convirtiendo la interfaz de una clase en otra que un cliente espera.	https://refactoring.guru/es/design-patterns/adapter	https://www.dofactory.com/javascript/design-patterns/adapter
Observer	Conductual	El patrón Observer define una dependencia de uno a muchos entre objetos, de manera que cuando uno cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.	https://refactoring.guru/es/design-patterns/observer	https://www.patterns.dev/vanilla/observer-pattern

Debes conocer

Los **patrones de arquitectura** son esquemas reutilizables que permiten diseñar la estructura global de una aplicación. Uno de los patrones de arquitectura más utilizados en desarrollo web es la arquitectura hexagonal. La arquitectura **hexagonal**, también conocida como **arquitectura de puertos y adaptadores**, es un enfoque de diseño de software que busca crear aplicaciones independientes de los frameworks y tecnologías utilizados para la entrada y salida de datos. Ofrece beneficios como una mayor modularidad, escalabilidad y mantenibilidad del código al separar las responsabilidades entre capas.

Puedes profundizar sobre la arquitectura hexagonal en:

- https://dev.to/dennysjmarquez/la-arquitectura-hexagonal-un-enfoque-para-desarrollar-aplicaciones-robustas-en-react-angular-y-vue-con-ejemplos-de-codigo-l4l

Para saber más

Puedes profundizar sobre patrones de diseño en:

- https://refactoring.guru/es/design-patterns/what-is-pattern
- https://kinsta.com/es/blog/patrones-de-diseno-javascript/
- https://www.patterns.dev
- https://www.dofactory.com/javascript/

Si estás interesado en libros el más recomendado es:

- Learning JavaScript Design Patterns: A JavaScript and React Developer's Guide

Y cursos online:

- Patrones de Diseño en JavaScript y TypeScript

4.1.- Componentes UI en JavaScript

Los patrones de diseño también se pueden aplicar al diseño de la interfaz de usuario. Para este propósito, el patrón de diseño más conocido es el basado en componentes.

El diseño basado en **componentes** implica dividir la interfaz de usuario en partes independientes y reutilizables llamadas componentes. Cada componente encapsula su propia estructura, comportamiento y estilo, lo que facilita su mantenimiento y reutilización en diferentes partes de la aplicación. Este enfoque permite construir interfaces de usuario de manera modular, donde cada componente puede ser desarrollado, probado y actualizado de manera independiente.

Un componente es una unidad independiente que puede contener HTML, CSS y JavaScript necesarios para renderizar una parte de la interfaz de usuario. Los componentes pueden ser simples, como un botón, o complejos, como un formulario o un modal. La encapsulación de funcionalidad y estilo dentro de componentes facilita la gestión del código y mejora la escalabilidad de la aplicación. Además, hay librerías de componentes UI en JavaScript y React

Los conceptos de encapsulamiento y clases, que hemos visto en esta unidad, son clave para el desarrollo web basado en componentes utilizado por los frameworks actuales. En futuras unidades profundizaremos sobre el uso de componentes en React y con solo JavaScript. Pero ya estamos listos para ver un pequeño ejemplo a forma de adelanto.

Ejemplo de componente con React

En React se definen componentes de una manera muy sencilla y clara. Aquí tienes un ejemplo básico de un componente de botón:

```
function MyButton() {
  function handleClick() {
    alert('¡Me hiciste clic!');
  }

  return (
    <button onClick={handleClick}>
      Hazme clic
    </button>
  );
}
```

Ejemplo de componente con JavaScript

Esto también se puede realizar utilizando solo JavaScript sin necesidad de ningún framework. Veamos un ejemplo. Este es algo complejo, pero lo revisitaremos en futuras unidades cuando tengamos un conocimiento más profundo del uso del DOM.

```
export default class MyButton {
    // Propiedad privada para almacenar el elemento raíz
    #root;
    #textoBoton;
    #avisoTexto;

    // Constructor de la clase, recibe el nodo donde se renderizará el botón
    constructor(targeNode,textoBoton="Hazme click",avisoTexto="Has hecho click!") {
      this.#root = targeNode;
      this.#textoBoton = textoBoton;
      this.#avisoTexto = avisoTexto;
      this.render();
    }

    // Método para renderizar el botón en el DOM
    render() {
        // Creamos un nodo de botón usando una función auxiliar
        let nodo=parseHTML(`
            <button>${this.#textoBoton}</button>
        `);

        // Asociamos un evento al botón para manejar clics
        nodo.addEventListener("click", this.handleClick.bind(this)); // El bind es necesario para no perder la referencia a la clase

        // Limpiamos el contenido actual del elemento raíz antes de añadir el nuevo botón
        while (this.#root.firstChild) {
            this.#root.removeChild(myNode.lastChild);
        }

        // Añadimos el nuevo botón al elemento raíz
        this.#root.appendChild(nodo);
    }
    // Método para manejar el evento de clic en el botón
    handleClick(event) { //Event guarda la información del evento
      alert(this.#avisoTexto);
      //ModalGenerator.newModal(this.#avisoTexto);
    }
}
```

Este componente solicita solamente que se le pase como referencia en el constructor un nodo, donde el componente se renderizara. También se le podría pasar información opcional como el aviso del botón. Esto se realizará en nuestro **main.js**:

```
//Importación
import MyButton from './MyButton.mjs'


//Inicialización
window.addEventListener("load",init);
function init(){
    let boton = new MyButton(
        document.getElementById("idBoton"),
        "Mi Botón",
        "Hola Mundo DWEC!!!"
    );
}
```

Debes conocer

También es posible tener una clase estática, ModalGenerator, con un método estatico newModal(text) que cada vez que se la llame genere un nuevo modal en el DOM.

JavaScript

```
export default class ModalGenerator {
    // Método estático para crear un nuevo modal
    static newModal(text) {
      // Crear el contenedor del modal usando parseHTML
      const modalContainer = parseHTML(`
        <div class="modal-container">
          <div class="modal-content">
            <p>${text}</p>
            <button class="close-button">Cerrar</button>
          </div>
        </div>
      `);

      // Añadir el evento de cierre al botón
      modalContainer.querySelector('.close-button').addEventListener('click', () => {
        document.body.removeChild(modalContainer);
      });

      // Añadir el modal al cuerpo del documento
      document.body.appendChild(modalContainer);
    }
  }

// Función auxiliar para convertir una cadena HTML en un nodo del DOM
// Es recomendable colocar esta función en un módulo externo, por ejemplo, Utils.mjs
function parseHTML(html) {
    var template = document.createElement("template");
    html = html.trim(); // Never return a text node of whitespace as the result
    template.innerHTML = html;
    return template.
}
```

CSS

```
.modal-container {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.5);
    display: flex;
    justify-content: center;
    align-items: center;
    z-index: 1000;
}

.modal-content {
    background: white;
```

```
            padding: 20px;
            border-radius: 5px;
            text-align: center;
            box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
        }

        .close-button {
            margin-top: 10px;
            padding: 10px 20px;
            background-color: #f44336;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        .close-button:hover {
            background-color: #d32f2f;
        }
```

Y ahora para genera una nueva alerta bastaría con:

```
//Importar la clase
import ModalGenerator from './ModalGenerator.mjs'

//Llamar al método estático
ModalGenerator.newModal("Hola Mundo!!!");
```

Por ejemplo, puedes hacer que el Modal se abra dentro de la función `handleClick()` del botón.

Para saber más

Estos componentes pueden ser encapsulados haciendo uso del Shadow DOM:

https://es.javascript.info/shadow-dom